CHANDIGARH UNIVERSITY

University Institute of Computing (UIC)

PROJECT REPORT
ON
AI-BASED RESUME SCREENING AND JOB MATCHING SYSTEM

Submitted in partial fulfilment for the award of the degree of
MASTER OF COMPUTER APPLICATION (MCA)

SUBMITTED BY:	SUBMITTED TO:
VIDUSHI (UID: 25MCI10158) **KHUSHI** (UID: 25MCI10383) Section: **25MAM-3A** MCA 2025 - 2027	**Dr. SARABJEET KAUR** Assistant Professor, UIC Chandigarh University Mohali, Punjab

ACADEMIC SESSION: JAN 2026 – MAY 2026
CHANDIGARH UNIVERSITY, MOHALI, PUNJAB

ACKNOWLEDGEMENT

We deem it a pleasure to express our heartfelt gratitude to our project guide, **Dr. Sarabjeet Kaur** (Assistant Professor, UIC), under whose valued guidance we have successfully carried out this project on **"AI-Based Resume Screening and Job Matching System"**. Her incisive and objective guidance, along with timely advice, provided us with constant motivation and encouragement throughout the design, algorithmic modeling, and implementation phases.

We also wish to express our sincere thanks to **Ms. Amanpreet Kaur Sandhu** (H.O.D, University Institute of Computing) for her valuable suggestions and academic encouragement, which significantly contributed to the successful completion of the project.

We are deeply grateful to **Dr. Manisha Malhotra**, Additional Director, University Institute of Computing, for providing a strong academic environment and fostering the disciplined, research-driven approach that enabled us to focus and dedicate ourselves to this work.

Finally, we acknowledge with deep appreciation the support, love, and encouragement of our parents and friends, without whom the completion of this project would not have been possible. Their patience and belief were instrumental in enabling us to overcome technical challenges and achieve our project goals.

DATED: April 2026
PLACE: Chandigarh University, Mohali, Punjab

SUBMITTED BY:
• **VIDUSHI** (UID: 25MCI10158)
• **KHUSHI** (UID: 25MCI10383)
Master of Computer Applications (MCA), Section: 25MAM-3A

INDEX

S. No.	Contents / Chapter Title	Page No.
1	Chapter 1: Introduction	1
2	Chapter 2: Objectives of the Project	2
3	Chapter 3: System Overview & Workflow	3 - 5
4	Chapter 4: System Architecture & Mathematical Formulation	6 - 7
5	Chapter 5: Tools and Technologies Used	8
6	Chapter 6: Skills & Competencies Utilized	9 - 10
7	Chapter 7: Results and Output Guide	11 - 13
8	Chapter 8: Advantages of the System	14
9	Chapter 9: Future Scope	15
10	Chapter 10: Conclusion	16
11	Chapter 11: Bibliography & References	17

CHAPTER 1: INTRODUCTION

In today's corporate recruitment landscape, human resource departments and talent acquisition teams receive hundreds or thousands of resumes for every advertised position. Manually reading, parsing, and screening this sheer volume of applications is time-consuming, expensive, and vulnerable to human fatigue and unconscious cognitive bias. Concurrently, students and job seekers face significant hurdles in tailoring their resumes to comply with modern Applicant Tracking Systems (ATS) and identifying concrete technical skill gaps before submitting applications.

To resolve these mutual challenges, the **AI-Based Resume Screening and Job Matching System** has been conceptualized and implemented. This project is an intelligent Natural Language Processing (NLP) and Machine Learning application designed to bridge the divide between job candidates and hiring organizations. Built upon advanced tokenization, lemmatization, named entity recognition (NER), and vector space modeling, the platform offers a unified, dual-workflow environment.

Key Highlights of the System:

- **Multi-format Document Ingestion:** Robust text extraction from PDF, DOCX, and TXT files with artifact removal.
- **Domain-Aware NLP Preprocessing:** Preserves crucial programming tokens (e.g. C++, .NET, CI/CD, Node.js) often lost during conventional punctuation stripping.
- **High-Precision Skill Phrase Extraction:** Leverages spaCy PhraseMatcher across an ontology of technical and soft skills.
- **Hybrid Multi-Factor Scoring:** Combines ontology-based Skill Overlap (55%), TF-IDF N-gram Cosine Similarity (35%), and Keyword Salience (10%).
- **Secure Dual-Mode Portal:** User Authentication and Career Preferences supporting Candidate ATS Optimization and Recruiter Batch Screening with CSV export.

CHAPTER 2: OBJECTIVES OF THE PROJECT

The primary objectives of this project are:

- To design and implement a secure, authenticated web application supporting Candidate and Recruiter roles with personalized career preferences.
- To build a universal document parser capable of accurately extracting clean text from PDF, DOCX, and TXT files.
- To engineer a domain-aware NLP preprocessor that performs case normalization, tokenization, lemmatization, and stopword removal without corrupting technical syntax.
- To extract candidate entity metadata (Name, Email, Phone, LinkedIn/GitHub links, Degrees, and Experience) using spaCy NER and pattern matching.
- To develop an ontology-backed Skill Extraction engine using spaCy PhraseMatcher to recognize multi-token technical phrases.
- To calculate mathematical similarity between resumes and job postings using TF-IDF sublinear scaling and Cosine Similarity in vector space.
- To generate real-time ATS match scores, missing skill checklists, and actionable resume optimization tips for candidates.
- To provide recruiters with a high-throughput batch screening pipeline that ranks applicants, generates Plotly radar charts, and exports results to CSV.

CHAPTER 3: SYSTEM OVERVIEW & WORKFLOW

The AI-Based Resume Screening and Job Matching System is architected with a modular, layered structure. It decouples presentation, authentication, NLP processing, and data persistence to ensure high reusability and maintainability.

3.1 Layered Architecture Overview

1. **Presentation Layer (Frontend):** Developed with Streamlit and custom CSS. Features authentication forms, job preference configuration cards, file uploaders, gauge meters, and radar visualizers.
2. **Application & Authentication Layer:** Manages user profiles using SHA-256 password hashing and persistent JSON storage. Automatically syncs user job preferences with active matching sessions.
3. **Core NLP Processing Layer:** Document extractor (pypdf, python-docx), NLP preprocessor (spaCy, NLTK), skill phrase matcher, entity metadata parser, and vector space similarity engine.

3.2 End-to-End System Workflow

Step 1: User accesses the portal and authenticates via Login or Register (or selects 1-Click Demo accounts).

Step 2: User configures Job Preferences (target job roles, seniority, work mode, and core technology stack).

Step 3: The system automatically adapts target job postings according to the selected preferences.

Step 4: In Candidate Mode, user uploads a resume or picks from 12+ pre-loaded resumes across diverse disciplines.

Step 5: In Recruiter Mode, recruiter uploads a job requisition and screens a batch of multiple resumes simultaneously.

Step 6: The text is extracted, normalized, stripped of extraction artifacts, and lemmatized.

Step 7: The hybrid matching engine computes composite scores, detects missing skills, and builds domain radar graphs.

Step 8: The UI displays interactive score gauges, candidate ranking leaderboards, and provides 1-click CSV download.

CHAPTER 4: SYSTEM ARCHITECTURE & MATHEMATICAL FORMULATION

4.1 Term Frequency - Inverse Document Frequency (TF-IDF)

TF-IDF evaluates the relative importance of words within a document corpus. Sublinear term frequency scaling is applied to prevent frequent technical terms from distorting scores:

$$\text{TF}(t, d) = 1 + \log(f(t, d)) \text{ if } f(t, d) > 0, \text{ else } 0$$

$$\text{IDF}(t, D) = \log\left((1 + |D|) / (1 + \text{DF}(t)) \right) + 1$$

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

4.2 Vector Space Cosine Similarity

The semantic similarity between preprocessed resume text vector (u) and job description vector (v) is computed as the cosine of the angle between them in high-dimensional vector space:

$$\text{Cosine Similarity}(u, v) = (u \cdot v) / (||u|| \times ||v||) = \sum(u_i \times v_i) / (\sqrt{(\sum u_i^2)} \times \sqrt{(\sum v_i^2)})$$

This metric yields a normalized score between 0.0 and 1.0 (scaled to 0 - 100%), invariant to document length.

4.3 Composite Multi-Factor Matching Formula

To balance semantic vocabulary with exact technical skill prerequisites, the platform employs a composite formula:

Composite Score = (w_skill × Skill Match %) + (w_tfidf × TF-IDF Similarity %) + (w_kw × Keyword Salience %)

Default calibrated weights: **w_skill = 0.55** (55% Skill Overlap), **w_tfidf = 0.35** (35% Semantic Vocabulary), **w_kw = 0.10** (10% High-Priority Keywords).

CHAPTER 5: TOOLS AND TECHNOLOGIES USED

Category	Technology / Tool	Role in Project
Programming Language	Python 3.12	Core backend, NLP pipeline, and mathematical logic
NLP Frameworks	spaCy 3.8.14 (en_core_web_sm)	PhraseMatcher for skills, Named Entity Recognition (NER)
Corpora & Lemmatization	NLTK 3.10	Tokenization, stopword removal, and WordNet lemmatizer
Machine Learning	Scikit-Learn 1.9	TfidfVectorizer, sublinear TF scaling, Cosine Similarity
Web Framework	Streamlit 1.65	Reactive web UI, multi-page routing, state management
Data Visualization	Plotly 7.1	Speedometer gauge charts, radar/spider charts, leaderboards
Document Parsers	pypdf 6.21 & python-docx 1.2	Text stream extraction from PDF and Word files
Document Generation	reportlab 5.1	Automated PDF resume and academic report generation
Data Handling	Pandas 3.0 & NumPy 2.5	Structured data aggregation, tabular ranking, CSV export
IDE & Environment	VS Code (Visual Studio Code)	Source code authoring, interactive terminal, debugging

CHAPTER 6: SKILLS & COMPETENCIES UTILIZED

• **Natural Language Processing:** Implemented tokenization, lemmatization, stopword removal, and specialized symbol preservation (C++, .NET, CI/CD).

• **Information Extraction:** Built spaCy NER pipelines and regex filters to extract candidate names, emails, phone numbers, and calculate experience years.

• **Vector Space Mathematics:** Developed TF-IDF feature matrices with unigram/bigram representation and computed Cosine Similarity angles.

• **Full-Stack Web Development:** Created reactive Streamlit interfaces with session state authentication, CSS styling, and Plotly data visualizations.

• **Software Quality Assurance:** Developed comprehensive unit tests with Python's unittest module, ensuring 100% test pass rate across the pipeline.

CHAPTER 7: RESULTS AND OUTPUT

The AI-Based Resume Screening and Job Matching System was extensively tested across both operational workflows:

1. **Authentication & Role Gate:** Secure login and registration portal featuring 1-click quick demo access for both Candidates and Recruiters.
2. **Job Preferences Setup:** Candidates configure target roles, seniority level, work mode, and target skills, automatically synchronizing matching sessions.
3. **Candidate ATS Optimizer:** Tested on resumes across 12 disciplines. Produces an instant ATS score gauge, color-coded badges for matched and missing skills, and domain coverage radar charts.
4. **Recruiter Batch Screening:** Evaluated on 18 candidate resumes simultaneously. Screens and ranks all candidates in under 3 seconds, displays a Plotly leaderboard, and exports full rankings to CSV.
5. **Test Suite Verification:** Executed 10 automated unit tests covering document parsing, preprocessing, entity extraction, TF-IDF cosine similarity, auth, and analytics with 100% success.

CHAPTER 8: ADVANTAGES OF THE SYSTEM

1. **Extreme Time Efficiency:** Reduces screening time from hours of manual review to seconds.
2. **Elimination of Human Bias:** Evaluates applicants objectively based on technical skill overlap and semantic vocabulary.
3. **Universal Document Support:** Natively ingests PDF, DOCX, and TXT documents without manual copying.
4. **High-Precision Phrase Matching:** Uses spaCy PhraseMatcher to avoid naive keyword counting errors.
5. **Actionable Candidate Feedback:** Highlights missing target skills and provides concrete tips to beat ATS parsers.
6. **Cost-Effective Architecture:** Built entirely on open-source Python libraries with no dependency on expensive paid APIs.
7. **Visual Analytics:** Provides clear Plotly charts (gauges, radar graphs, leaderboards) accessible to non-technical users.

CHAPTER 9: FUTURE SCOPE

- **Transformer Semantic Embeddings:** Integrating Sentence-BERT (SBERT) or HuggingFace Transformers for deep contextual similarity.
- **Optical Character Recognition (OCR):** Adding Tesseract OCR to parse scanned image-based PDF resumes.
- **Automated Resume Re-Writing with GenAI:** Leveraging LLMs to automatically rephrase bullet points using the Google X-Y-Z formula.
- **Direct ATS & HRIS Integration:** Connecting to corporate HR platforms (Workday, Greenhouse, Taleo) via REST APIs.
- **Multilingual Support:** Extending the spaCy NLP pipeline to support French, German, Spanish, and Hindi resumes.
- **AI Mock Interview Simulation:** Generating technical interview questions tailored to detected resume skill gaps.

CHAPTER 10: CONCLUSION

The **AI-Based Resume Screening and Job Matching System** demonstrates the practical application of Natural Language Processing, machine learning algorithms, and modern web engineering to address real-world recruitment challenges.

By synthesizing multi-format document extraction, domain-aware tokenization, spaCy phrase matching, TF-IDF vectorization, and composite scoring, the system delivers an objective, rapid, and transparent evaluation pipeline. It simultaneously functions as an invaluable ATS optimizer for job candidates and an indispensable batch ranking engine for corporate recruiters.

All functional and academic objectives set forth for this project have been fully achieved, establishing a rock-solid, production-grade foundation for ongoing advancements in AI-assisted recruitment.

CHAPTER 11: BIBLIOGRAPHY & REFERENCES

[1] I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.

[3] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, Stanford University, 2024.

[4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[5] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, CNNs and incremental parsing," 2017. [Online]. Available: https://spacy.io/

[6] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[8] Streamlit Inc., "Streamlit Documentation and Development Guides," [Online]. Available: https://docs.streamlit.io/

[9] Python Software Foundation, "The Python Standard Library Documentation," [Online]. Available: https://docs.python.org/3/

[10] Plotly Technologies Inc., "Plotly Open Source Graphing Library for Python," [Online]. Available: https://plotly.com/python/

[11] pyPDF Community, "pypdf Documentation," [Online]. Available: https://pypdf.readthedocs.io/

[12] python-docx Community, "python-docx Documentation," [Online]. Available: https://python-docx.readthedocs.io/